MOTION DETECTION SECURITY MONITORING SYSTEM

CSC 126 – Graphics and Visual Computing | Short Project Documentation

Submitted by: Cafe, Princess Shayna O.
Cago, Andrea Marx C.
Consigna, Rowella J.

Section: BJY21
Date: 10/13/26

Submitted to: Joshua Tubo

1. Project Description

The Motion Detection Security Monitoring System (MDSMS) is a Python-based computer vision project that uses OpenCV to detect movement in a live webcam feed. It processes camera frames, compares changes between frames, identifies moving regions, and displays the detected areas using bounding boxes and a motion-status label. The notebook also includes security-event logging that saves snapshots and records event timestamps in a CSV file, along with a dashboard for viewing monitoring information.

2. Project Objectives

- To capture frames from a webcam and process them using OpenCV.
- To apply grayscale conversion and Gaussian blur to prepare frames for motion analysis.
- To detect changes between consecutive frames and identify moving regions.
- To visualize detected motion with bounding boxes and status labels.
- To record motion events and save snapshots for later review.

3. Problem / Real-World Use-Case

A basic security-monitoring setup may require a person to watch a camera continuously. This project demonstrates a simple way to highlight visible movement and keep a record of detected events. When movement is detected, the application can display a motion alert, save an image snapshot, and write event information to a CSV log. This can help a user review activity, but it is a demonstration system rather than a complete or professionally validated security product.

4. Techniques Used

- **Image operations:** The captured frame is converted from BGR color to grayscale using cv2.cvtColor(). Gaussian blur using a 21 × 21 kernel reduces small image variations before comparison.
- **Image segmentation:** The absolute difference between the previous and current grayscale frames is calculated. Thresholding separates significant pixel changes from smaller changes, and dilation helps make changed regions more visible.

- **Motion detection and analysis:** cv2.findContours() extracts contours from the thresholded image. Regions smaller than the configured minimum contour area of 1,500 pixels are ignored. Bounding boxes are drawn around qualifying regions, and the status is set to indicate whether motion was detected.

5. Screenshots of Results

This section presents screenshots of the application during testing. Each screenshot should be taken from the actual running application.

5.1 Live Webcam Capture



Figure 1. Live Webcam Capture — Shows the live webcam feed used to monitor the surrounding area in real time.

5.2 Image Preprocessing



Figure 2. Image Preprocessing — Shows the preparation of captured video frames for motion detection.

5.3 Motion Detection



Figure 3. Motion Detection — Shows the processed frame used to identify movement in the monitored area.

5.4 Motion Detection with Bounding Boxes

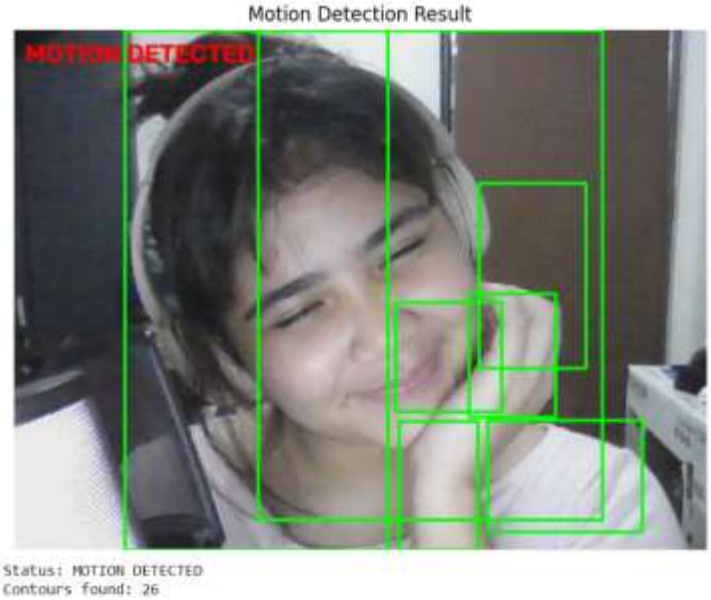


Figure 4. Motion Detection with Bounding Boxes — Shows a detected moving object highlighted in the live webcam feed.

5.5 Motion Detection Event Log

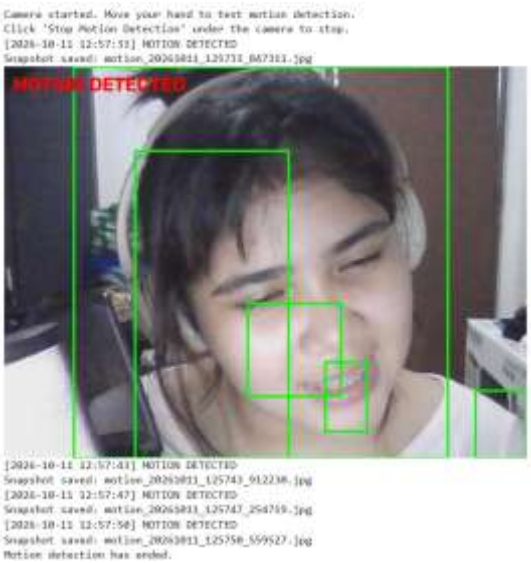




Figure 5. Motion Detection Event Log — Shows the recorded motion events and their available details.

7. Observed Behavior and Accuracy Discussion

In the output, the detection cell reported “MOTION DETECTED” and 26 contours found. The live-monitoring output also records motion events and messages confirming that snapshot files were saved. These outputs show that the detection and event-recording workflow ran during the captured test. The contour count is not an accuracy score it represents the number of contours found in that particular frame. No formal accuracy percentage is reported because the it does not contain a labeled test dataset, ground-truth annotations, or a calculation of precision, recall, or false-positive rate. The project is evaluated here through observed behavior in the saved run rather than a statistical accuracy claim.

8. Limitations

- Lighting changes, shadows, or camera movement may be detected as motion even when no relevant object is moving.
- The minimum contour-area threshold can cause small movements to be ignored, while larger background changes may trigger detection.
- The system detects changes in image pixels; it does not recognize a person's identity or determine whether an activity is a real security threat.
- The project uses Google Colab and browser-based webcam access, so it depends on camera permissions and the Colab environment.
- Event logging and dashboard behavior depend on the cells and their execution order. The dashboard should be tested during the final demonstration.

9. Conclusion

The Motion Detection Security Monitoring System demonstrates how classical computer vision techniques can be combined for a basic monitoring task. Grayscale conversion, Gaussian blur, frame differencing, thresholding, contour extraction, and bounding-box visualization are used to identify visible movement. The project also includes event snapshots, CSV logging, and a monitoring dashboard. The saved run shows motion detection and snapshot logging, while further testing under varied conditions is needed to assess reliability and reduce false detections.